



Discrete Optimization

Multiple linear ordering problem

Marc Benito-Marimon , Rafael Martí , Anna Martínez-Gavara *

Universitat de València, Departament d'Estadística i Investigació Operativa, C/Doctor Moliner, 50, Burjassot, 46100, Valencia, Spain

ARTICLE INFO

Keywords:

Multiple optimal solutions
Linear ordering problem
Variable neighborhood search
Metaheuristics

ABSTRACT

The linear ordering problem (LOP) is a well-known NP-hard combinatorial optimization problem. Given a set of elements and its dominance matrix, the objective is to find a simultaneous ordering of rows and columns of the matrix, to maximize the sum of entries above the main diagonal. This linear ordering returns a rank of the elements maximizing the dominance, which provides a practical tool in many applications from economy to social networks. However, the LOP may have many alternate optima with significantly different orderings, which makes it difficult to provide an optimal representative ranking. In this paper, we study the rankability of the input matrix in terms of the similarities among alternate optimal rankings. We propose new mathematical models to formulate the rankability as an optimization problem. We also present a heuristic algorithm, based on the VNS methodology, to target large instances. Our computational testing shows that the analysis of several alternate optimal solutions may provide the user with practical information to overcome the limitations of a single optimal ranking.

1. Introduction

The Linear Ordering Problem (LOP) consists of finding a permutation p of the columns (and rows) of an $n \times n$ matrix of weights $C = (c_{ij})$, to maximize the sum of the weights in the triangle above the main diagonal.

In its graph version, the LOP is defined as follows. Let $D = (V, A)$ denote the complete weighted digraph on n nodes, i.e., the directed graph with node set $V = \{1, 2, \dots, n\}$, arc set A and the property that for every pair of nodes i and j there is an arc $(i, j) \in A$ from i to j with weight c_{ij} and an arc $(j, i) \in A$ from j to i with weight c_{ji} . A tournament T in A consists of a subset of arcs containing for every pair of nodes i and j either arc (i, j) or arc (j, i) , but not both. An acyclic tournament is a tournament without directed cycles, i.e., not containing an arc set of the form $\{(v_1, v_2), (v_2, v_3), \dots, (v_k, v_1)\}$ for some $k > 1$ and distinct nodes $v_1, v_2, \dots, v_k$. For the remainder of the paper, we use v_i or i interchangeably to define a node without loss of generality.

A linear ordering of the nodes $V = \{1, 2, \dots, n\}$ is a ranking of the nodes given as linear sequence, or equivalently, as a permutation of the nodes. An acyclic tournament T in A corresponds to a linear ordering of the nodes in V . The LOP consists of finding a linear ordering, or acyclic tournament T , with maximum value of $f(T)$:

$$f(T) = \sum_{(i,j) \in T} c_{ij}. \quad (1)$$

The LOP has also been formulated in terms of permutations. As described above, a solution can be interpreted as a permutation $p = (p_1, \dots, p_n)$, where p_i represents the element in position i . Therefore, the objective function value of a solution p can be computed from the matrix of weights $C = (c_{ij})$ as:

$$f(p) = \sum_{i=1}^{n-1} \sum_{j=i+1}^n c_{p_i p_j}.$$

The LOP can be then stated as finding the permutation p with maximum $f(p)$ value. It can be also formulated with the following linear integer model with binary variables x_{ij} , for $i, j = 1, \dots, n$ where $x_{ij} = 1$ if i precedes j and $x_{ij} = 0$ if j precedes i . We assume $i \neq j$.

$$\begin{aligned} \max f(x) &= \sum_{(i,j) \in A} c_{ij} x_{ij} \\ x_{ij} + x_{ji} &= 1, \text{ for } i, j = 1, \dots, n, i < j, i \neq j, \\ x_{ij} + x_{jk} + x_{ki} &\leq 2, \text{ for } i, j, k = 1, \dots, n, i < j, i < k, j \neq k, \\ x_{ij} &\in \{0, 1\}, \text{ for all } i, j = 1, \dots, n. \end{aligned} \quad (2)$$

This 0/1-IP can be considered as the canonical IP formulation of the problem (Martí & Reinelt, 2022). The basic observation in terms of graphs, is that in an ordering or permutation exactly one of the arcs (i, j) or (j, i) is present for every pair of nodes i and j , and that a tournament is acyclic if and only if it does not contain any dicycle of length 3

* Corresponding author.

E-mail addresses: marc.benito@uv.es (M. Benito-Marimon), Rafael.Marti@uv.es (R. Martí), gavara@uv.es (A. Martínez-Gavara).URL: <https://www.uv.es/gavara/> (A. Martínez-Gavara)

(which justifies the inclusion of the second family of constraints in the formulation above).

Linear ordering problems may occur whenever rankings of some objects are to be determined. For example, given a set of n objects $O_1, O_2, \dots, O_n$ to be rated by m persons according to their individual preferences. A ranking of these objects is to be found which reflects these single rankings as closely as possible, and its application is indeed the oldest one of the LOP, proposed in 1959 by Kemeny (1959). This problem is solved as an LOP by setting c_{ij} equal to the number of persons preferring object O_i to object O_j .

The most important application of the LOP is probably the triangulation of *input-output tables* in economy. That was the central component of the study for which Leontief (1986) was awarded the Nobel Prize in 1973. These tables represent the dependencies between the different branches of an economy divided into sectors, where the entries in the table show the transactions between sectors. Triangulation is a method for determining a hierarchy of all sectors such that the amount of flow incompatible with this hierarchy (i.e., from sectors ranked lower to sectors ranked higher) is as small as possible. Such rankings allow interpretations of the industrial structure of a country and comparisons between different countries.

A recent LOP application can be found in the context of ranking US colleges. The US News and World Report ranks each year US colleges according to some of their features. A dominance matrix can be created from some characteristics of the colleges, such as graduation rate, student/faculty ratio, percentage of classes under 20 students or acceptance rate. The entry c_{ij} in this matrix is the number of features for which college i is scored higher than college j . The LOP optimal solution provides an overall ranking summarizing all the features considered, and therefore is an excellent source of information for prospective students and their families. Colleges in the upper part of the ranking are considered better than those ranked in posterior positions. As a matter of fact, universities promote programs to climb positions in these rankings as a way to attract students. However, as pointed out by Anderson et al. (2022), the existence of alternate optimal LOP solutions with different associated rankings brings up the issue of fairness. When two rankings are equally optimal, which one should be used? The problem arises when these optimal rankings are significantly different. In this paper, we study the alternate optimal LOP solutions as a means to recommend the user (student in our example) its suitability.

Cameron et al. (2021) proposed an optimization model to compute the two LOP optimal rankings at maximum distance, in an effort to disclose if the LOP provides a fair or representative optimal solution. The authors applied their model to several examples from sports to college rankings. These two previous papers on multiple optimal LOP solutions (Anderson et al., 2022; Cameron et al., 2021), triggered our interest on this challenging problem. We found that they constitute an important theoretical contribution, but the user of the LOP solution could need further information to decide whether to apply or not an LOP optimal solution. We study this problem to find additional characteristics to help the final user in practical terms. Specifically, the contributions of our paper are:

- We extend the mathematical model to obtain several optimal rankings at maximum inter-distance, to provide the user with a more representative set of alternate optimal LOP solutions.
- We propose a new objective function to better model the inter-distance between rankings.
- We complement the model with a heuristic algorithm to solve medium and large size instances.
- We provide the user with the range of the elements' positions in the optimal rankings as a practical tool for usage.
- We perform extensive experimentation to compare our models and methods with previous approaches.

2. Literature review

As shown above, the analysis of multiple optimal solutions has important applications. However, it has received very little attention in the LOP literature. According to the recent monograph in the LOP (Martí & Reinelt, 2022), the main contribution in this line has been recently published. In particular, Anderson et al. (2022) were the first to study the examination of alternate optimal LOP solutions to conclude that some elements are relatively stable across different optimal solutions but others rank in very different positions, which jeopardizes the applicability of the LOP. To illustrate this point, the authors show the following 10 optimal solutions of the N-pa113 instance (called *Paley graph*) taken from the public domain LOLIB - Linear Ordering LIBrary (Martí et al., 2010).

1. $\langle 8, 9, 3, 12, 7, 1, 10, 4, 13, 2, 11, 5, 6 \rangle$
2. $\langle 8, 9, 3, 12, 6, 1, 10, 4, 13, 7, 2, 11, 5 \rangle$
3. $\langle 8, 9, 3, 12, 1, 10, 4, 13, 7, 2, 11, 5, 6 \rangle$
4. $\langle 9, 12, 1, 10, 4, 13, 7, 8, 2, 11, 5, 3, 6 \rangle$
5. $\langle 9, 12, 7, 1, 10, 4, 13, 8, 2, 11, 5, 3, 6 \rangle$
6. $\langle 8, 11, 9, 3, 12, 6, 1, 10, 4, 13, 7, 2, 5 \rangle$
7. $\langle 8, 11, 9, 3, 12, 6, 7, 1, 10, 4, 13, 2, 5 \rangle$
8. $\langle 8, 9, 3, 12, 6, 7, 1, 10, 4, 13, 2, 11, 5 \rangle$
9. $\langle 9, 3, 12, 7, 1, 10, 4, 13, 8, 2, 11, 5, 6 \rangle$
10. $\langle 9, 3, 12, 1, 10, 4, 13, 7, 8, 2, 11, 5, 6 \rangle$

These optimal solutions can be easily obtained with the *pool mode* of a commercial software like Gurobi or CPLEX. However, as will be shown in our computational testing, in most of the cases a complete enumeration of all the optimal solutions is not possible due to its very large cardinality.

It is clear from the optimal solutions above, that element 9 is consistently ranked in the first positions and the element 5 in the last positions. However, there are other elements, such as 3 or 6 that rank in very different positions, depending on the optimal solution considered. In particular, in Solution 1, element 3 ranks in position 3, but in Solution 4 it ranks in position 12. Similarly, element 6 ranks in position 5 in Solution 2, but it ranks in the last position in many other optimal solutions. Considering that most of the solving methods proposed for the LOP only identified one optimal solution (or simply good in the case of heuristics), we have to be careful when considering the individual positions of the elements, especially in some applications. That is especially true for those applications in which we want to obtain a ranking of the objects or elements for which we have pairwise comparisons, such as, for example, ranking in sports tournaments or lists of best universities. In these cases, the LOP seems not to be an appropriate model by itself, and require additional elements to disclose the most adequate ranking.

An early measure to study the rankability of a matrix is given by the so-called *degree of linearity* (Martí & Reinelt, 2022). This value, λ , quantifies how well an LOP optimal solution represents the dominance in the matrix data. Given a matrix $C = (c_{ij})$, it is defined in mathematical terms as:

$$\lambda(C) = \frac{c^*}{\sum_{i \neq j} c_{ij}} \quad (3)$$

where c^* is the optimum LOP value. In other words, $\lambda(C)$ is the ratio between the maximum sum of the upper-triangle and the total sum of the elements in the matrix. This means that when a matrix has a high degree of linearity (i.e., $\lambda(C)$ is close to 1), the sum of the elements of the matrix in the upper-triangle (c^*) is significantly larger than the sum in the lower part, when ordered according to an optimal solution.

Given an optimal solution (ranking), the degree of linearity can be interpreted as the percentage of pairs of elements aligned with it in the data matrix. Ideally speaking, $\lambda(C) = 1$ would indicate that all the pairs of elements are aligned (well represented) with the optimal ranking. On the contrary, $\lambda(C) = 0.5$ indicates that an optimal ordering is not meaningful since there are many pairs that do not agree with the ranking (i.e.,

their corresponding element in the upper diagonal is lower than their element in the lower diagonal). We would say in that case that the data matrix is not properly rankable. When applied to economics, the degree of linearity is a measure of the development of an economy, where a high developed economy, typically will have a low degree, indicating high circulation of goods among sectors and an underdeveloped economy will show a high degree of linearity, denoting a hierarchy among sectors.

From a theoretical perspective, it is difficult to establish the relationship between the number of ties in the matrix $c_{ij} = c_{ji}$ and the degree of linearity. This is mainly because it depends on the scale of the input data values. As described in [Martí and Reinelt \(2022\)](#), to make computations independent of the scale, LOP instances are typically normalized, where each c_{ij} value is replaced with $c_{ij} - \min\{c_{ij}, c_{ji}\}$. In this way, ties in the original matrix translate into zero values in the normalized matrix, which leads to a higher degree of linearity. We have experimentally confirmed this point by computing it in the 50 I/O normalized instances ([Martí et al., 2010](#)). These 50 real-world instances constitute the standard benchmark in LOP experiments. They compute the relationships among n economical sectors of different countries (where n ranges from 44 to 79) and their values range from 0 to 20,000. [Fig. 1](#) illustrates the correlation between the percentage of repeated elements (ties) depicted in the x -axis and the degree of linearity (y -axis). In all these instances, the degree of linearity is relatively high (between 0.93 and 0.99) probably due to the large number of 0s (ranging from 52.7 % to 85.73 %), and we can observe a certain trend in which the higher the percentage of ties in the data matrix, the higher the degree of linearity.

[Anderson et al. \(2022\)](#) present another method to gather information of the optimal solution set without computing it. They define the matrix X^f where the entry (i, j) is the percentage of optimal solutions in which the element i ranks before j . Unfortunately, the computational cost of obtaining X^f is very high, so the authors propose the use of the LOP root relaxation (solved with an interior point method), X^* , as an approximation to it.

The existence of multiple optimal solutions is related to the weights in the data and an empirical analysis on the LOLIB reveals that many instances have more than one optimal solution. [Anderson et al. \(2022\)](#)

propose an exact method to obtain all the optimal solutions for small instances, and some of the medium size ones. Additionally, they identify some important (extreme) optimal solutions, that may help to obtain a final ranking. In particular, they consider the centroid of the set of optimal solutions, and the nearest and farthest solutions from the centroid. This study has obviously important implications in the application of the LOP and opens very interesting areas of research to enrich this model.

In terms of the representativeness of an optimal ordering for an LOP instance, it is interesting to study the distance between alternate optimal solutions. If we can find two optimal solutions with their elements in very different positions (i.e., if the solutions are at a large distance), then we cannot consider any of them as representative. On the contrary, if all the optimal solutions are very similar (i.e., if they are at a close distance to each other), then we can consider any of them as a good representative. In line with that, [Cameron et al. \(2021\)](#) and [Anderson et al. \(2022\)](#) proposed the use of the Kendall-tau distance developed by Maurice Kendall (1938) to compare rankings and permutations, and identified the two optimal solutions at the largest distance. Specifically, given two permutations p and p' , $K_d(p, p')$ denotes the Kendall-tau rank distance computing the number of pairs of elements with different relative order in p and p' .

The authors proposed a two-step method in which they first solve the canonical LOP to find the optimal value c^* , and then solve a mathematical model to maximize the distance between all the pairs of optimal LOP solutions. The model in [Cameron et al. \(2021\)](#) follows.

In the canonical LOP formulation (2), the $n \times n$ dimensional binary variable x represents the optimal solution. In particular, x_{ij} takes the value 1 if element i precedes j in the optimal ordering. In the new model, [Cameron et al. \(2021\)](#) introduce another $n \times n$ binary variable y indicating a second optimal solution. To obtain the two farthest optimal solutions, the objective function to minimize is defined as the sum of z -variables, where each z_{ij} takes the value 1 if elements i and j are in the same relative positions in solutions x and y , as measured by $x_{ij} + y_{ij} - z_{ij} \leq 1$. The rest of the constraints in model (4) are directly taken from the canonical model together with an additional constraint to guarantee the optimality of x and y (i.e., with an objective function

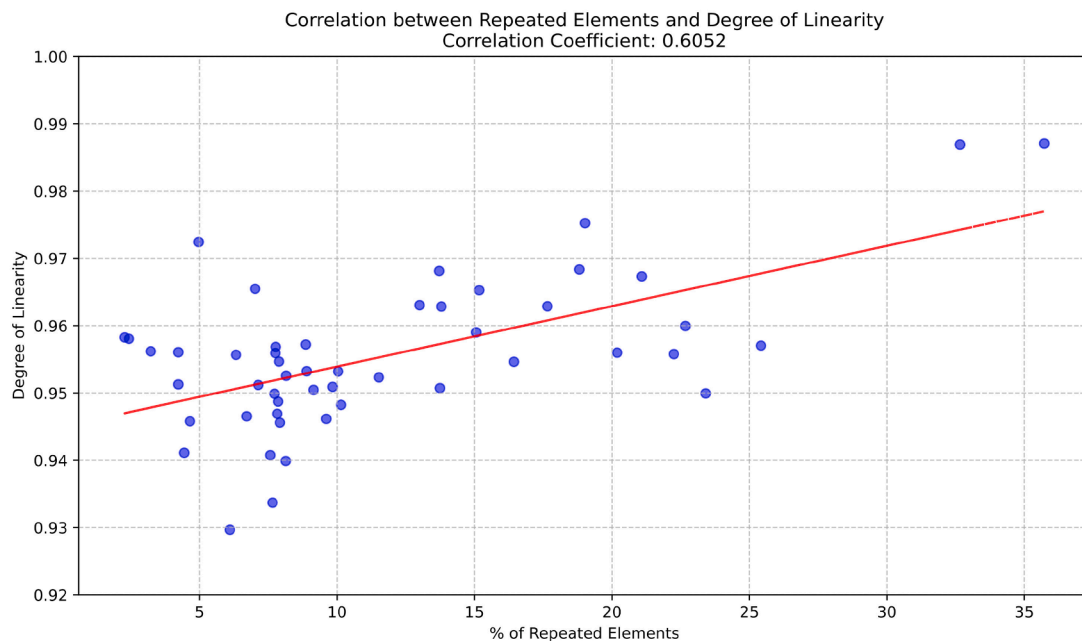


Fig. 1. Degree of linearity on the y -axis and the percentage of tied elements on the x -axis.

value equal to c^*).

$$\begin{aligned}
 \min \quad & \sum_{i \neq j: i, j \in \{1, \dots, n\}} z_{ij} \\
 \text{s.t.} \quad & x_{ij} + x_{ji} = 1, \quad \forall i < j: i, j \in \{1, \dots, n\}, \\
 & y_{ij} + y_{ji} = 1, \quad \forall i < j: i, j \in \{1, \dots, n\}, \\
 & x_{ij} + x_{jk} + x_{ki} \leq 2, \quad \forall i < j, i < k, j \neq k: i, j, k \in \{1, \dots, n\}, \\
 & y_{ij} + y_{jk} + y_{ki} \leq 2, \quad \forall i < j, i < k, j \neq k: i, j, k \in \{1, \dots, n\}, \\
 & \sum_{i \neq j: i, j \in \{1, \dots, n\}} c_{ij} x_{ij} = c^*, \\
 & \sum_{i \neq j: i, j \in \{1, \dots, n\}} c_{ij} y_{ij} = c^*, \\
 & x_{ij} + y_{ij} - z_{ij} \leq 1, \quad \forall i \neq j: i, j \in \{1, \dots, n\}, \\
 & x_{ij}, y_{ij}, z_{ij} \in \{0, 1\}, \quad \forall i \neq j: i, j \in \{1, \dots, n\},
 \end{aligned} \quad (4)$$

In this paper, we focus on this aspect of the problem, the representativeness of an optimal solution with respect to the set of all optimal solutions. We view our work as a generalization of the approach by Cameron et al. (2021) in two different ways. On the one hand, we will explore if the addition of more diverse optimal solutions could be of any help when applying an optimal ranking to a decision problem (like for example the selection of a university college based on rankings). On the other hand, we will explore the applicability of this model for medium and large size instances. Our empirical experimentation will show that, as expected, these instances pose a challenge to mathematical models, and thus we complement our study with the proposal of a heuristic, based on the Variable Neighborhood Search methodology to obtain an efficient approximation to this problem.

3. New mathematical models

We build our work from the previous mathematical model (4) by Cameron et al. (2021). We find interesting and useful to identify the two optimal solutions at the largest distance, but at the same time, we think that it is worth it to investigate if the information provided by these two extreme solutions would be enough to help the decision maker to apply the LOP optimal solution. On the one hand, we have experimentally found that in many instances we cannot obtain all the optimal solutions due to its cardinality (larger than half a million in some cases) which makes challenging to disclose if one optimum may represent them all. In addition to the computational difficulty to obtain all of them, we have to consider how to show it to the user. On the other hand, we want to disclose if finding more than 2 diverse optimal solutions would help the decision maker more than having just two of them.

As mentioned, the number of alternate optimal solutions is usually very large. For example, in the LOLIB set of instances, we can find the Input-Output subset, in which the instances range from $n = 44$ to 56. In spite of the medium size of these instances, we have found that the size of the set of optimal solutions is very large (ranging in most of the cases from 1000 to 500000 solutions). This seems to indicate that in a way or another we have to simplify this information to be practical to the user. In this way, the proposal of Cameron et al. (2021) and Anderson et al. (2022) of obtaining the 2 most distant seems reasonable. However, we wonder if this would be enough to provide a significant summary of the set of optimal solutions. The following example indicates that it would be better to obtain 3 or 4 optimal solutions at the largest distance.

Consider again the 10 optimal solutions of the N-pal13 instance. The two solutions at the largest distance computed with the model in Cameron et al. (2021) are:

$$\begin{aligned}
 & \langle 9, 12, 1, 10, 4, 13, 7, 8, 2, 11, 5, 3, 6 \rangle \\
 & \langle 8, 11, 9, 3, 12, 6, 7, 1, 10, 4, 13, 2, 5 \rangle
 \end{aligned}$$

The examination of these two extreme optimal solutions provides some useful information about the ranking of the elements across multiple optimal solutions. For example, element 7 ranks in the 7-th position

in both solutions, while element 8 ranks in position 8-th in the first solution, and in the first position in the second solution. We can represent as $[7, 7]$ the range of element 7, and $[1, 8]$ the range of element 8. The main question for the user would be, are these ranges representative of the positions that these two elements, 7 and 8, occupy across all the optimal solutions? We can easily see that the answer is no. The following three optimal solutions show that element 7 may rank in positions 3, 7, and 10, and therefore the information provided by the two most distant solutions above can be misleading in terms of the position of its elements.

$$\begin{aligned}
 & \langle 8, 9, 3, 12, 6, 1, 10, 4, 13, 7, 2, 11, 5 \rangle \\
 & \langle 8, 11, 9, 3, 12, 6, 7, 1, 10, 4, 13, 2, 5 \rangle \\
 & \langle 9, 12, 7, 1, 10, 4, 13, 8, 2, 11, 5, 3, 6 \rangle
 \end{aligned}$$

We now generalize the mathematical model in (4) to identify the more than two farthest optimal solutions. Let's say that we want to identify the m most distant optimal LOP solutions (in terms of the Kendall-tau distance). We define an $n \times n$ dimensional binary variable x^t for each optimal solution, $t \in \{1, \dots, m\}$. As in the previous formulation, x^t_{ij} takes the value 1 if element i precedes j in the t optimal solution. For each couple (t, t') with $t \neq t'$, of optimal solutions x^t and $x^{t'}$, the variable $z^{(t,t')}_{ij}$ takes the value 1 if elements i and j are in the same relative positions in solutions x^t and $x^{t'}$. This is thanks to the constraint $x^t_{ij} + x^{t'}_{ij} - z^{(t,t')}_{ij} \leq 1$, as in the previous model. To obtain the m farthest optimal solutions, the objective function minimizes the sum of $z^{(t,t')}$ -variables. The rest of the constraints in the model are directly generalized from (4).

$$\begin{aligned}
 \min \quad & \sum_{t, t'=1}^m \sum_{i, j=1}^n z^{(t,t')}_{ij} \\
 \text{s.t.} \quad & x^t_{ij} + x^t_{ji} = 1, \quad \forall i < j: i, j \in \{1, \dots, n\}, \quad \forall t \in \{1, \dots, m\} \\
 & x^t_{ij} + x^t_{jk} + x^t_{ki} \leq 2, \quad \forall i < j, i < k, j \neq k: i, j, k \in \{1, \dots, n\}, \quad \forall t \in \{1, \dots, m\} \\
 & \sum_{i, j=1}^n c_{ij} x^t_{ij} = c^* \quad \forall t \in \{1, \dots, m\} \\
 & x^t_{ij} + x^{t'}_{ij} - z^{(t,t')}_{ij} \leq 1, \quad \forall i \neq j: i, j \in \{1, \dots, n\} \forall t \neq t': t < t', t, t' \in \{1, \dots, m\} \\
 & x^t_{ij}, z^{(t,t')}_{ij} \in \{0, 1\}, \quad \forall i \neq j: i, j \in \{1, \dots, n\}, \forall t \neq t': t < t', t, t' \in \{1, \dots, m\}
 \end{aligned} \quad (5)$$

It is clear that the model (5) may have a very large number of variables, since for each couple of optimal solutions that we want to identify, it introduces an $n \times n$ dimensional binary variable $z^{(t,t')}$. On the other hand, from a practical point of view, we will consider relatively small values of the number of optimal distant solutions m .

We now present an alternative model to find distant optimal LOP solutions based on the studies on diversity maximization. In particular, Martí et al. (2022) and Parreño et al. (2021) concluded that when maximizing the diversity among m selected points from a larger set, the model based on maximizing the minimum distance, called MaxMin, generates solutions with a very different structure than the model based on maximizing the sum of distances, MaxSum. Specifically, it obtains equidistant points all over the space, and it does not avoid to select points in central parts as the MaxSum model does. We call m -farthest-sum to the model in (5), and propose a second model based on maximizing the minimum distance among optimal solutions, called m -farthest-min, to find the m most distant solutions. It is worth mentioning that a downside of the model based on the sum of distances, is that it may obtain the same optimal solution several times (i.e. it may repeat some of the solutions) when maximizing the overall sum of distances. The model based

on maximizing the minimum distance does not have this issue, because the repetition of a solution would result in a minimum value of 0.

Note that our models are based on computing with the variable z_{ij} the number of pairs (i, j) in the same relative order in alternate optimal solutions (say t and t'). Therefore, to maximize the distance among solutions, they minimize the sum of pairs of elements with the same order in the solutions. In particular, the m -farthest-sum model minimizes the following expression in the objective function of (5):

$$\sum_{\substack{i,j=1 \\ i \neq j}}^n z_{ij}^{(t,t')},$$

which counts the number of pairs (i,j) in the same relative position in solutions t and t' . The objective function adds this expression for all pairs of solutions, thus obtaining:

$$\sum_{\substack{t, t'=1 \\ t < t'}}^m \sum_{\substack{i, j=1 \\ i \neq j}}^n z_{ij}^{(t, t')}.$$

To minimize this expression is equivalent to maximize the Kendall-tau distance between solutions. We formally define the mathematical IP formulation of the m -farthest-min model as:

$$\begin{aligned}
\min \quad & z \\
\text{s.t.} \quad & x_{ij}^t + x_{ji}^t = 1, & \forall i < j : \quad \begin{matrix} i, j \in \{1, \dots, n\}, \\ \forall t \in \{1, \dots, m\} \end{matrix} \\
& x_{ij}^t + x_{jk}^t + x_{ki}^t \leq 2, & \forall i < j, \quad \begin{matrix} i < k, j \neq c : i, j, \\ k \in \{1, \dots, n\}, \\ \forall t \in \{1, \dots, m\} \end{matrix} \\
& \sum_{\substack{i,j=1 \\ i \neq j}}^n c_{ij} x_{ij}^t = c^* & \forall t \in \{1, \dots, m\} \\
& x_{ij}^t + x_{ij}^{t'} - z_{ij}^{(t,t')} \leq 1, & \forall i \neq j : \quad \begin{matrix} i, j \in \{1, \dots, n\} \forall t \neq t' : \\ t < t', t, t' \in \{1, \dots, m\}, \end{matrix} \\
& z \geq \sum_{\substack{t,j=1 \\ t \neq j}}^n z_{ij}^{(t,t')} & \forall t \neq t' : t < t', t, t' \in \{1, \dots, m\} \\
& x_{ij}^t, z_{ij}^{(t,t')} \in \{0, 1\}, & \forall i \neq j : \quad \begin{matrix} i, j \in \{1, \dots, n\}, \forall t \leq t' : \\ t < t', t, t' \in \{1, \dots, m\}. \end{matrix}
\end{aligned}
\tag{6}$$

In the m -farthest-min model presented above, the right-hand-side of the new constraint

$$z \geq \sum_{\substack{i,j=1 \\ i \neq j}}^n z_{ij}^{(t,t')},$$

counts the number of pairs of elements in the same relative ordering in solutions t and t' . With the minimization of the z variable in the objective function, it takes in this constraint the value of the largest number of elements with the same ordering in a pair of solutions (t, t') . Note that this pair corresponds to the two solutions at minimum distance (since they are the two with more pairs in common).

4. Algorithmic approach

As mentioned in the introduction, our goal is to inform users about how the position of each element varies within the set of optimal rankings. To achieve this, in the previous section, we first solve the linear ordering problem (LOP) in terms of permutations to obtain the objective function value c^* , and subsequently we try to find m solutions with c^* as objective function value that maximize the Kendall-tau distance

among them. In this section, we consider the use of a heuristic to this end, to be able to solve large size instances. It's important to note that, due to the nature of employing a heuristic to address the LOP problem, the optimality of the resulting set of solutions cannot be guaranteed. As it is customary in heuristic optimization, we propose an algorithm to approximate the optimal solutions, which in our case translates into the range of positions for the elements in the optimal rankings.

We propose a two-step algorithm based on the Variable Neighborhood Search (VNS) methodology . In Algorithm 1, we show the pseudocode of our complete approach. The algorithm takes five inputs: the number of m alternative solutions that we want to obtain, the largest neighborhood indices $k_{\max}^1$ and $k_{\max}^2$, the maximum number of iterations $it_{\max}$, and the maximum CPU time $\tau_{\max}$. As in the mathematical models proposed above, the heuristic algorithm has basically two phases. In the first one, the algorithm finds the best solution (i.e., its approximation to the optimal LOP solution) with the BVNS method , numbered as step 2 in the pseudo-code. In the second phase (numbered as step 3), it finds the subset of the m farthest solutions corresponding to the value identified in the first phase, which will ultimately determine the elements' ranking (computed in step 4). We denote as Δ^S the set of rankings of the elements in the m alternative solutions obtained.

Algorithm 1 two-step VNS($m, k_{max}^1, it_{max}, k_{max}^2, \tau_{max}$).

1: $p \leftarrow \text{random_solution}()$	
2: $p \leftarrow \text{BVNS}(p, k_{\max}^1, it_{\max})$	▷ see Algorithm 2
3: $S \leftarrow \text{VNS}(p, m, k_{\max}^2, \tau_{\max})$	▷ see Algorithm 3
4: $\Delta^S \leftarrow \text{elements_range}(S)$	
5: return Δ^S	

VNS was introduced in [Mladenović and Hansen \(1997\)](#) and, since then, has gained significant relevance among metaheuristics. Several variants and extensions have been proposed, as detailed in [Hansen et al. \(2017\)](#). Starting from an initial solution, the rationale behind VNS is to exploit the notion of changing neighborhoods, both in reaching the local optimum and in escaping from the attraction basins that contain them. This paper considers the neighborhoods based on insertion moves proposed in [Garcia et al. \(2006\)](#). Given a solution (permutation) $p = (p_1, \dots, p_n)$, where p_i represents the element in position i in the solution p , we define as $\text{insert}(p_j, i)$ dropping element p_j from solution p and adding it in position i with $j \neq i$. Then, the neighborhood $N_1(p)$ consists of the set of solutions obtained by the moves $\text{insert}(p_j, j-1) \forall j \in \{2, \dots, n\}$ and $\text{insert}(p_j, j+1) \forall j \in \{1, \dots, n-1\}$, which essentially represents the set of solutions obtained by switching one element either directly one position before or after its current position. The neighborhood $N_2(p)$ is defined as the solutions obtained by inserting any element p_j from p into a different position. In general, we define $N_k(p)$ as the set of solutions reached by applying N_2 recursively to solution p , $k-1$ times.

In the **first step** of our algorithm, we employ a Basic VNS (BVNS) method to solve the LOP, to obtain the objective function value of a high-quality solution that eventually can be the optimal solution. BVNS balances diversification and intensification by integrating deterministic and random changes within its neighborhood structures. [Algorithm 2](#) shows the pseudo-code of the BVNS.

The BVNS algorithm takes three inputs: a randomly generated initial solution, the largest neighborhood index k_{max}^1 , and the maximum number of iterations it_{max} . At each iteration it , the algorithm first sets the parameter k to 1 (step 2). The main loop then iterates through neighborhoods (steps 3–12). First, a solution p' is randomly generated from the k^{th} neighborhood of solution p (step 4). A local search is applied to the obtained solution p' to find a local optimum p'' in the neighborhood $N_2(p')$ (step 5) with a best-improving strategy. Specifically, the local search iterates through each element (p'_j) in p' , inserting it into the position that yields the best objective function value. If the improved solution p'' outperforms the current solution p , it becomes the new in-

Algorithm 2 BVNS($p, k_{\max}^1, it_{\max}$).

```

1: while  $it < it_{\max}$  do
2:    $k \leftarrow 1$ 
3:   while  $k \leq k_{\max}^1$  do
4:      $p' \leftarrow Shake(p, k)$ 
5:      $p'' \leftarrow LocalSearch(p')$ 
6:     if  $f(p'') > f(p)$  then
7:        $p \leftarrow p''$ 
8:        $k \leftarrow 1$ 
9:     else
10:       $k \leftarrow k + 1$ 
11:   end if
12: end while
13: end while
14: return  $p$ 

```

cumbent solution $p \leftarrow p''$, and the neighborhood exploration restarts at the smallest index $k \leftarrow 1$. Otherwise, the algorithm explores a larger neighborhood ($k = k + 1$) until reaching the largest one. Note that in case of ties in the evaluation within neighboring solutions, the method selects the first solution found.

As mentioned above, the **second step** of our algorithm aims to find the m farthest alternative solutions according to the *Kendall tau distance*, which quantifies the dissimilarity between two permutations. Given two solutions (permutations) p and p' of length n , the Kendall tau distance is denoted as $K_d(p, p')$. Intuitively, the Kendall tau distance counts the number of inversions needed to convert one permutation into another. An *inversion* is a pair of elements in the opposite order in the two permutations. Given a subset of m solutions $S = \{p^1, \dots, p^m\}$, the Kendall tau distance of the set S , $K_d(S)$, is computed as the sum of the Kendall tau distance of each pair of solutions in S . In our VNS algorithm, a subset S' is better than a subset S if it has a larger Kendall tau distance, i.e., $K_d(S') > K_d(S)$.

Algorithm 3 outlines the pseudo-code for computing the m farthest solutions. It takes the following inputs: the best solution found in the BVNS, p ; the number m of desired farthest alternative solutions; the maximum size of the neighborhood $k_{\max}^2$; and the maximum CPU time $\tau_{\max}$.

The algorithm first initializes the objective function value of the best solution found p (step 1) and the set S of alternative high-quality solutions (step 2). It then enters in a loop that continues until the time limit $\tau_{\max}$ is reached. Inside the loop, a solution q is randomly selected from the set S (step 5). The main loop then iterates based on a basic variable neighborhood search scheme (steps 6–18). The shake procedure is applied to randomly select a solution p' in the neighborhood $N_k(q)$ (step 7). Then, a local search is performed on p' to find a local optimum within its N_2 neighborhood (step 8).

If the new solution p'' matches our best value, then we proceed with the construction of S' . If the set S already contains m elements, then the set S' is created by replacing one solution in S with the new solution p'' in a way that maximizes the overall gain in Kendall tau distance. If the set S has less than m solutions, S' is created by simply adding p'' to S . Finally, if the new set S' improves the overall Kendall distance compared to S , then S is updated with S' , and the neighborhood index is reset to 1 (step 13). Otherwise, the neighborhood index is incremented by 1 (step 15). Once the m farthest solutions are obtained, the range of the elements' positions is computed, as detailed in the complete algorithm (Algorithm 1). It is worth mentioning that Algorithm 3 disregards solutions that improve the LOP objective function value of the BVNS solution (Algorithm 2), as its aim is to identify diverse and distant alternative solutions.

Algorithm 3 VNS ($p, m, k_{\max}^2, \tau_{\max}$).

```

1:  $c \leftarrow f(p)$ 
2:  $S \leftarrow \{p\}$ 
3: while  $\tau < \tau_{\max}$  do
4:    $k \leftarrow 1$ 
5:    $q \leftarrow select\_random(S)$ 
6:   while  $k < k_{\max}^2$  do
7:      $p' \leftarrow Shake(k, q)$ 
8:      $p'' \leftarrow LocalSearch(p')$ 
9:     if  $f(p'') == c$  then
10:       $S' \leftarrow Insert(S, p'', m)$ 
11:      if  $K_d(S') > K_d(S)$  then
12:         $S \leftarrow S'$ 
13:         $k \leftarrow 1$ 
14:      else
15:         $k \leftarrow k + 1$ 
16:      end if
17:    end if
18:  end while
19: end while
20: return  $S$ 

```

Table 1 collects the main symbols and definitions introduced above.

5. Computational experiments

This section describes the computational experiments that we performed to test the effectiveness and efficiency of the mathematical models and the algorithm proposed. In particular, we consider the two mathematical models m-farthest-sum and m-farthest-min with $m = 2, 3, 4$ and 5 alternate optimal solutions, proposed in Section 3, and the heuristic approach VNS, proposed in Section 4. We first describe the test instances in our experimentation, then the statistics applied to

Table 1
Symbols and definitions.

Symbol	Definition
D	Digraph: $D = (V, A)$
V	Set of vertices of D
A	Set of arcs of D
$C = (c_{ij})$	Matrix data of D
c^*	optimal objective function value
n	number of elements (vertices)
x_{ij}	binary variable which takes the value 1 if element i precedes j in the optimal ordering
p	solution of LOP ($p_1, \dots, p_n$)
p_i	is the i -th element of solution p
g_i	is the position of the element i
S	set of solutions $\{p^1, \dots, p^m\}$
$\mathcal{P}$	set of optimal solutions
$ \mathcal{P} $	number of optimal solutions
$\lambda(C)$	degree of linearity

Table 2
Number of optimal solutions in 10 instances.

Instance	n	$ P $
N-stabu74	50	90720
N-t59d11xx	44	> 500000
N-t65b11xx	44	> 500000
N-t65i11xx	44	> 500000
N-t70l11xx	44	> 500000
N-t70x11xx	44	8
N-t75e11xx	44	516
N-tiw56n54	56	6384
N-tiw56n66	56	2016
N-usa79	79	144

summarize the results, and then we report on the output of our methods, both models and heuristic. Special attention is paid to the comparison with the results of the previous model (Cameron et al., 2021) that obtains two optimal distant solutions. All the experiments have been performed in an AMD Ryzen 9 5950x (3.4 GHz) with 16GB RAM using C++ as programming language. The source code is publicly available at <https://github.com/opticom-hub/Multiple-LOP/>.

5.1. Instances

Martí and Reinelt (2022) compiled a set of benchmark instances of the Linear Ordering Problem called the LOLIB. These problem instances consist of real-world as well as randomly generated data sets. The most applied subset in the LOLIB is the so-called Input-Output (I/O) tables. These are 50 real-world data sets taken from input-output tables from various sources, typically the economical sectors of different countries. The original entries in these tables were not necessarily integral, but they were scaled to integral values.

I/O tables include European sector tables compiled by the Statistical Office of the European Communities (Eurostat), Belgian tables, German tables compiled by the Deutsches IW and by the Statistisches Bundesamt, and input-output tables for the economy of the United States made available by Knuth (1993).

As stated by Martí and Reinelt (2022), the 50 instances in the I/O set of the LOLIB are relatively easy to solve for nowadays methods, both exact and heuristics. Branch and cut methods are able to obtain the optimal solution of all these instances and certify their optimality, and modern metaheuristics can identify them in very short computational times. However, they have not ever considered to study alternate optimal solutions. Considering that we know their optimal value, and one optimal solution, they constitute an appropriate test-bed to study the problem to identify all their optimal solutions, that we call here the multiple LOP.

In our first experiment, we want to compute the number of alternate optimal solutions in the I/O set of the LOLIB, in order to evaluate the size of the search space in our problem. Tables 2 and 3 show the results of this experiment. Table 2 reports on 10 representative instances in this set, showing the name of the instance (Instance), its size (n), and the number of alternate optimal solutions, $|P|$, obtained with the *pool mode* of Gurobi. This table shows that in some instances, such as N-t59d11xx, the number of optimal solutions is extremely large. In particular, we limit Gurobi to store 500,000 alternate optimal solutions, which is exceeded in this instance. On the contrary, we may also find some instances, such as N-t70x11xx with only 8 alternate optimal solutions. Table 3 summarizes this information for the entire I/O set, reporting the range on the number of alternate optimal solutions, $|P|$, the number of instances in each subset, #instances, and the average size (Avg. n).

Table 3 shows that from the set of 50 I/O instances, 21 of them exhibit a large number of alternate optimal solutions (more than 500,000), while only 7 of them have less than 500 optimal solutions. In general terms, we may say that I/O instances present many alternate optimal solutions, and then it makes sense to study the similarity among them in an effort to establish the representativeness of a given one. This ta-

Table 3
Number of optimal solutions in the I/O instances.

$ P $	#instances	Avg n	Avg $\lambda(C)$
[1,500)	7	44	0.949
[500,10000)	11	50	0.954
[10000,500000)	11	53.45	0.952
> 500000	21	45.33	0.957

ble also shows that the size of an instance (its number of elements to rank), is not related (in statistical terms, does not explain) its number of optimal solutions. Note that the last row in the table reports on the set with the largest number of optimal solutions, and it has an average size of 45.33, while the previous row, in which the number of alternate optimal solutions is between 10,000 and 500,000 has an average size of 53.45.

Fig. 2 summarizes information about the size, the number of optimal solutions, and the degree of linearity (indicated by dot sizes) of the I/O instances. At first glance, we could say that instances with larger sizes tend to have a large number of optimal solutions. However, we cannot compare them with small instances since they present a wide range in the number of optimal solutions (i.e., we cannot guarantee that instances with a small size have fewer optimal solutions). In fact, some of the small size instances ($n = 44$) present more optimal solutions than some of the large instances ($n = 60$). We did not find any clear pattern connecting the degree of linearity with the instance size and number of optimal solutions. This may be partially explained by the small range of degree of linearity values in this set of instances.

As described in Section 2, the number of ties in the data matrix may have implications in different aspects of the associated LOP instance. In particular, we study in Fig. 3 the relationship between the number of ties and the number of optimal solutions. The graphic reveals a significant correlation between these variables, which aligns with theoretical expectations. This correlation can be explained by the presence of ties in the dominance matrix between adjacent elements in an optimal ranking. Each such tie introduces an additional optimal solution. When these ties occur in multiple positions, their combined effect can significantly increase the total number of optimal solutions, as illustrated in Fig. 3. For example, given an optimal solution $p = (p_1, \dots, p_i, p_{i+1}, \dots, p_n)$ if $c_{p_i, p_{i+1}} = c_{p_{i+1}, p_i}$ then it is clear that the permutation $p' = (p_1, \dots, p_{i+1}, p_i, \dots, p_n)$ has the same objective function value than p , and therefore is also an optimal solution.

It must be noted that, for the sake of simplicity, we depict in Fig. 3 that some of the instances have 500,000 optimal solutions. Their exact number of optimal solutions is indeed larger than this value, but we stopped counting them due to its computational cost.

Note that when an instance has a large number of optimal solutions, it does not necessarily mean that an optimal solution does not represent well the entire set; or in other words, that an individual ranking would be useless. The objective of our study is not only to quantify the size of the set of optimal solutions, but to investigate how these solutions compare among them. In particular, we propose to identify the variability of the elements in this optimal set. It must be noted that in some cases two alternate optimal solutions may be very similar, and only vary in the inversion of pairs of elements, leading to a large number alternate relatively similar solutions. Our experiments in the next subsections will disclose it.

5.2. Mathematical models

In this work, we propose a novel way of representing the set of optimal solutions P for the LOP. Instead of the traditional way of reporting the optimal solutions (rankings), we propose to show the range of positions in which each element varies across P . This will provide information about the similarity among optimal solutions, and therefore the representativeness of one of them. However, explicitly computing this

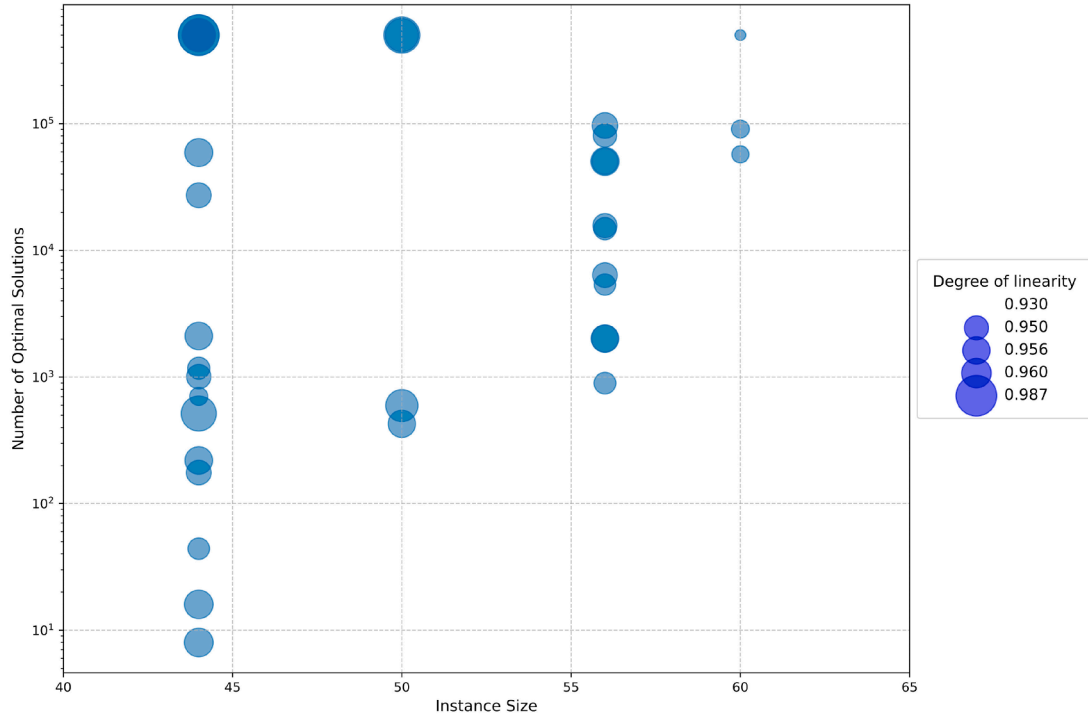


Fig. 2. Degree of linearity (dot size) in terms of instance size (x-axis) and number of optimal solutions (y-axis).

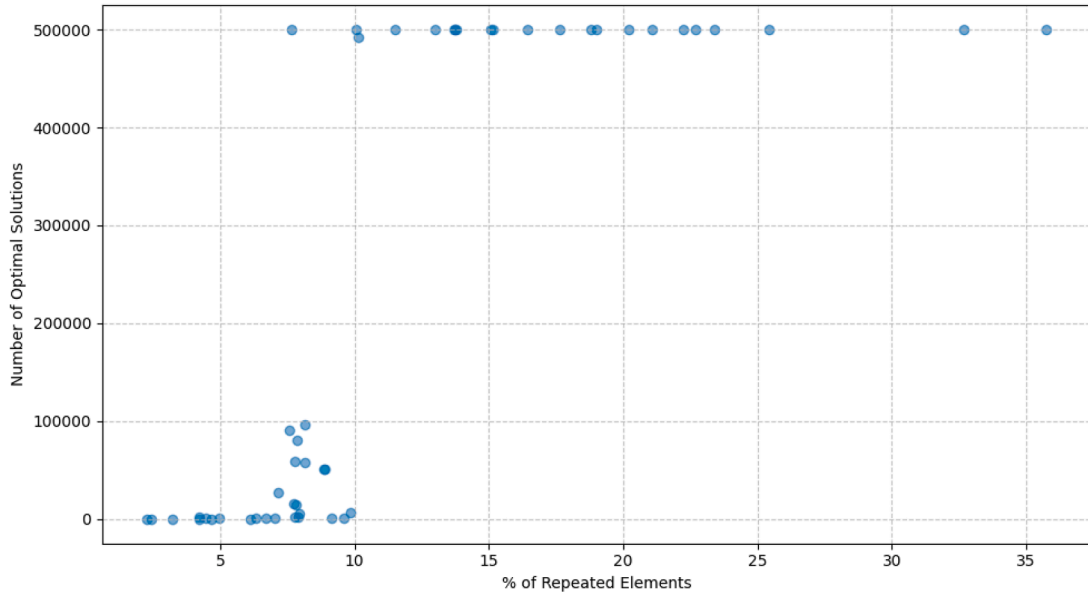


Fig. 3. Number of optimal solutions with respect to percentage of ties.

set can be challenging because of its size. Hence, we propose two ways to approximate these ranges, a first one applying mathematical models, and a second one applying a heuristic. To measure how well a subset of optimal solutions represent the entire set $\mathcal{P}$, we employ the following metrics:

We define the positions range of an element i in a subset of solutions S as

$$\Delta_i^S = [a, b], \quad (7)$$

where a is the minimum (best) position of element i across the set S , and b is the maximum (worst) position. Then, the width of that range is

$$|\Delta_i^S| = b - a \quad (8)$$

From these definitions, $\Delta_i^{\mathcal{P}} = [a, b]$ provides the range of positions in which element i can be found when considering all the optimal solutions. In terms of rankings, the minimum position a provides its best rank, while the maximum position b gives the worst one. This would be a very useful information to the user interested on how well ranks element i ; however, as mentioned above, it is difficult to calculate since it requires the previous computation of the set $\mathcal{P}$. We therefore will approximate it from a subset S , and compute the accuracy metric δ to evaluate this approximation as:

$$\delta = \frac{\sum_{i=1}^n |\Delta_i^S|}{\sum_{i=1}^n |\Delta_i^{\mathcal{P}}|} \quad (9)$$

Table 4
Instances selected for the experiments.

Instance	n	$ P $
N-be75np	50	596
N-t65w11xx	44	220
N-t70d11xx	44	704
N-t70f11xx	44	1008
N-t70w11xx	44	16
N-t70x11xx	44	8
N-t74d11xx	44	176
N-t75d11xx	44	44
N-t75e11xx	44	516
N-t75i11xx	44	1176
N-tiw56r54	56	896
N-usa79	79	144

Consider an example in which we know the entire set of optimal solutions P , and element $i = 1$ occupies positions 3, 4, and 8 overall these optimal rankings. Then, we say that $\Delta_1^P = [3, 8]$. Consider that we obtain a subset S containing three optimal solutions, in which element 1 occupies positions 3, 3, and 4, respectively. Then, $\Delta_1^S = [3, 4]$. The question is: Does S provide a good approximation of P in terms of the rankings of the elements? The δ value helps us to answer this question by comparing the sum of $|\Delta_i^S|$ with the sum of $|\Delta_i^P|$ for all the elements i . In mathematical terms, we say that δ ranges in the $[0, 1]$ interval, and values closer to 1 indicate a better approximation than values closer to 0.

Note that since $|\Delta_i^S| \leq |\Delta_i^P|$ for all i , then a measure of the quality of the approximation of the exact ranges in P by S is given by the maximum of the differences between these two values across the elements. In mathematical terms,

$$\text{MaxDiff} = \max\{|\Delta_1^P| - |\Delta_1^S|, \dots, |\Delta_n^P| - |\Delta_n^S|\} \quad (10)$$

The approximate ranges in S are computed from the solutions' set resulting from our mathematical models described in Section 3, and the exact ranges from the whole set of optimal solutions obtained by brute force with Gurobi and the classic LOP formulation. In this experiment, we only use a subset of the I/O instances where we are able to compute all the optimal solutions in a reasonable amount of time (in 2 hours of CPU time). In Table 4 we present the size and number of optimal solutions of the selected instances. Table 5 reports the results of the exact range Δ^P and the approximation range Δ^S obtained with the mathematical model, *m-farthest-sum* described in Section 3 and solved with Gurobi. In particular, we apply this model to obtain $m = 2, 3$, and 4 optimal solutions at the largest distance in the N-pal13 instance. This table reports the Δ , $|\Delta|$, and MaxDiff values for this instance.

The first column in Table 5 shows the list of 13 elements in the instance N-pal13. Columns two and three show the range of the positions Δ_i^P and its width $|\Delta_i^P|$ for each element i respectively. The following columns show these two statistics for the solution of the *m-farthest-sum* with m equal to 2, 3, and 4 respectively. This particular instance has 10 optimal solutions. This table also shows the average values across all the elements in the last three rows.

To understand the results in Table 5, we have to compare the ranges obtained with the exact solution with those obtained with the mathematical models. For example, we can observe that the element 1 ranks between the third position and the eighth position in the exact solution (i.e., with the examination of all the optimal solutions), while it ranks between positions 4 and 8 in the three optimal solutions obtained with the *m-farthest-sum* model with $m = 3$. In the bottom row Avg. Δ_i , we can see that the exact solution has an average width of ranks of 5.69 while this model ($m = 3$) has an average of 5.13. Comparing this statistic for different models, we may conclude that the model with $m = 4$ improves upon the model with $m = 2$. This is very relevant because one of the contributions of this paper is the extension of the model with $m = 2$ to models with larger values of m . The δ values are in line with this observation, concluding that the model with $m = 4$ is the best one,

with results very close to the exact method based on the examination of the entire set P .

Table 6 shows the results obtained with the *m-farthest-sum* model on the 12 I/O instances for which we can explicitly compute the entire set of optimal solutions P . As in the previous experiment, this table reports the statistics δ and MaxDiff to compare the range of each element in the optimal solutions obtained with the model, with the range in the entire set P . For example, in the first row of Table 6 we see that the model with $m = 2$ computes the ranges of the N-be75np instance with a precision $\delta = 0.8$, and the largest error that this model makes is of 2 positions (MaxDiff=2). When computing five optimal distant solutions, that is with $m = 5$, the model obtains a $\delta = 0.9$ of accuracy and MaxDiff=2, which is significantly better than with $m = 2$. The instances N-t74d11xx and N-t65w11xx exhibit a remarkable $\delta = 1$ and MaxDiff=0 obtained with the model with $m = 4$, showing that it is possible to obtain the entire range of all elements across all the optimal solutions without computing the entire set P , from only four distant optimal solutions in these cases. There are some instances in which the model with $m = 2$ is able to match the entire range for all elements but, it is important to note that these coincide with those that have the lowest number of optimal solutions. That is the case, for example, of N-t70w11xx and N-t70x11xx. However, there are other instances, such as N-t74d11xx and N-t65w11xx mentioned above, in which the models with $m = 2$ and $m = 3$ are not able to match the entire range, and the models with larger values of m provide a better approximation of their ranges.

In the bottom row of Table 6, we report the average values of δ and MaxDiff across the 12 instances. These statistics seem to indicate that the larger the m -value in the model, the better the results, thus showing that it is worth it to consider more than 2 optimal solutions to summarize the ranking results of an instance. To further analyze this point, we conducted a Wilcoxon signed rank test comparing the δ values with $m = 2$ and with a larger value of m . We choose the largest value in this experiment, $m = 5$, for this pairwise test. The p -value obtained is 0.01125, which supports our thesis. However, the increase in the accuracy comes with a price in the computational times. The *m-farthest-sum* model has very low CPU times with $m = 2$ (between 10 and 20 seconds) while the CPU model with $m = 3$ ranges from 14 to 400 seconds. Larger values of m present larger times, reaching in some cases the 10 minutes of CPU time. Note that the explicit computation of P to obtain the complete range of all the elements would take more than 2 hours in some instances, which makes convenient the application of the *m-farthest-sum* model.

To investigate the impact of the number of global optima on the performance of the models, we categorized the 12 instances used in our experiments into three groups based on their number of optimal solutions: Small (≤ 150 global optima), Medium (between 151 and 700), and Large (> 700). Fig. 4a reports the average δ values obtained for each group and value of m , while Fig. 4b shows the corresponding average MaxDiff values, which quantify the structural variability among the selected solutions.

As can be seen in Fig. 4a, for instances in the Small group, all values of m lead to high δ values, indicating consistently high-quality and diverse solution sets. However, as the number of optimal solutions increases, the benefits of larger m values become more evident. In the Medium and Large groups, models with $m = 4$ and $m = 5$ outperform those with lower m values, suggesting more robustness and effectiveness in capturing well-spread solutions when the set of optima is larger and more heterogeneous.

Fig. 4b complements this observation by illustrating how the structural differences among selected solutions evolve with m and instance group. For Medium and Large instances, higher m values result in notably lower MaxDiff values, indicating more stable and cohesive selection of solutions. These results support the idea that a larger value of m improves the sampling of diverse optimal solutions, particularly when the number of optima is large.

To complement the previous analysis, Fig. 5 presents scatter plots showing the relationship between the number of global optima and the

Table 5

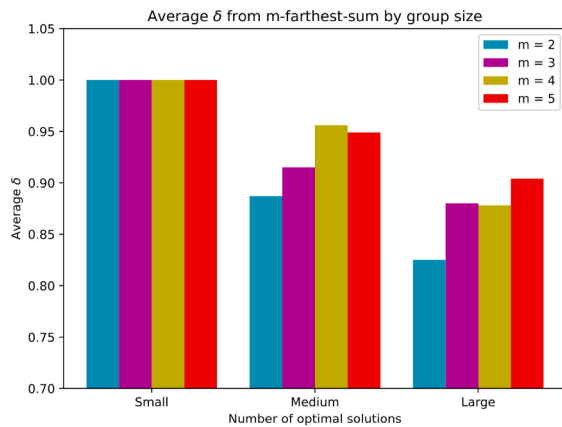
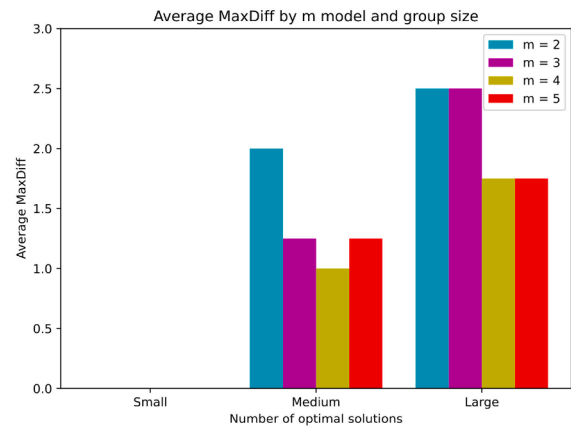
Position of the elements in the optimal rankings of the m -farthest-sum model on the N-PAL13 instance.

Element (i)	Exact solution		$m = 2$		$m = 3$		$m = 4$	
	Δ_i^P	$ \Delta_i^P $	Δ_i^S	$ \Delta_i^S $	Δ_i^S	$ \Delta_i^S $	Δ_i^S	$ \Delta_i^S $
1	[3, 8]	5	[3, 8]	5	[4, 8]	4	[3, 8]	5
2	[9, 12]	3	[9, 12]	3	[9, 12]	3	[9, 12]	3
3	[2, 12]	10	[4, 12]	8	[3, 12]	9	[4, 12]	8
4	[5, 10]	5	[5, 10]	5	[6, 10]	4	[5, 10]	5
5	[11, 13]	2	[11, 13]	2	[11, 13]	2	[11, 13]	2
6	[5, 13]	8	[6, 13]	7	[5, 13]	8	[6, 13]	7
7	[3, 11]	8	[7, 7]	0	[3, 10]	7	[3, 11]	8
8	[1, 9]	8	[1, 8]	7	[1, 8]	7	[1, 8]	7
9	[1, 3]	2	[1, 3]	2	[1, 3]	2	[1, 3]	2
10	[4, 9]	5	[4, 9]	5	[5, 9]	4	[4, 9]	5
11	[2, 12]	10	[2, 10]	8	[2, 12]	10	[2, 10]	8
12	[2, 5]	3	[2, 5]	3	[2, 5]	3	[2, 5]	3
13	[6, 11]	5	[6, 11]	5	[7, 11]	4	[6, 11]	5
Avg. Δ_i		5.69		4.61		5.13		5.23
MaxDiff		0		8		1		2
δ		1		0.81		0.90		0.92

Table 6

Results of the m -farthest-sum model in the instances with optimal set $\mathcal{P}$ known.

Instance	$m = 2$		$m = 3$		$m = 4$		$m = 5$	
	δ	MaxDiff	δ	MaxDiff	δ	MaxDiff	δ	MaxDiff
N-be75np	0.8	2	0.82	1	0.95	2	0.9	2
N-t65w11xx	0.915	2	0.957	1	1	0	1	0
N-t70d11xx	0.915	2	0.936	2	0.979	2	0.989	1
N-t70f11xx	0.733	4	0.733	4	0.867	1	0.733	4
N-t70w11xx	1	0	1	0	1	0	1	0
N-t70x11xx	1	0	1	0	1	0	1	0
N-t74d11xx	0.956	2	0.978	1	1	0	0.989	1
N-t75d11xx	1	0	1	0	1	0	1	0
N-t75e11xx	0.875	2	0.906	2	0.875	2	0.906	2
N-t75i11xx	0.778	2	0.889	2	0.778	2	0.917	1
N-tiw56r54	0.873	2	0.96	2	0.889	2	0.976	1
N-usa79	1	0	1	0	1	0	1	0
Avg.	0.903	1.5	0.928	1.25	0.944	0.916	0.950	1

(a) Average δ value.

(b) Average MaxDiff values.

Fig. 4. Average δ and MaxDiff values for different values of m , grouped by the number of optimal solutions per instance..

two key metrics in our study: δ and MaxDiff, for the instances in Table 4. In Fig. 5a, a negative correlation is observed between the δ metric and the number of global optima, indicating that as the number of optima increases, the selected solutions become more differentiated within the solution set. This trend is more noticeable for smaller values of m , as shown by the steeper regression lines. Conversely, Fig. 5b shows a positive correlation between MaxDiff and the number of global optima.

The fitted regression lines highlight that a larger set of optima is associated with greater structural variability among the selected solutions, especially for lower values of m .

Table 7 reports for the m -farthest-min model the same two statistics considered in Table 6. We can observe a similar pattern than in the m -farthest-sum model, increasing the average δ metric when increasing m .

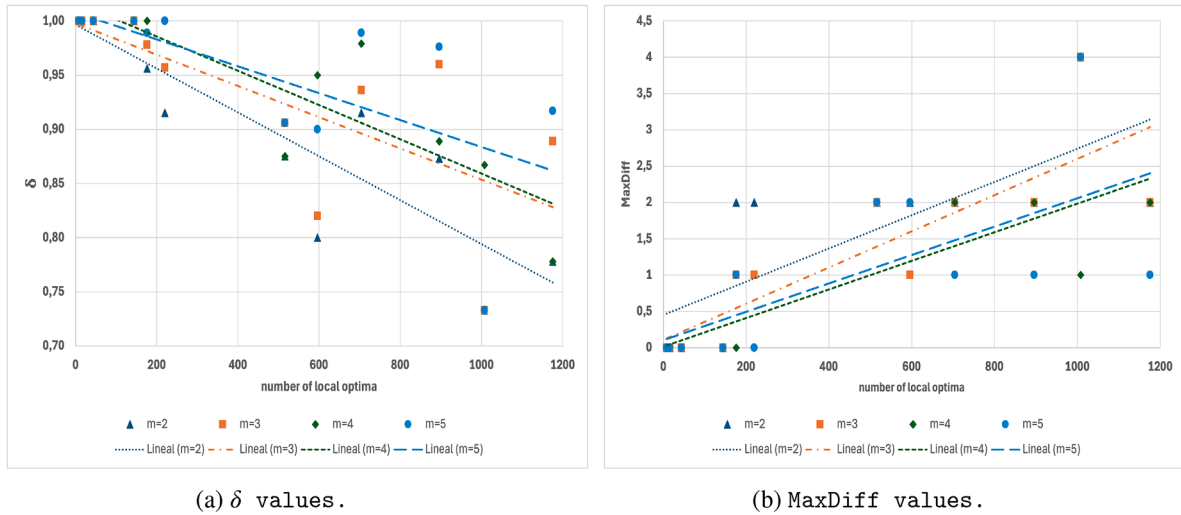


Fig. 5. Scatter plot of the number of optimal solutions per instance against δ (left) and MaxDiff values (right) for different values of m .

Table 7

Results of the m -farthest-min model in the instances with optimal set $\mathcal{P}$ known.

Instance	$m = 2$		$m = 3$		$m = 4$		$m = 5$	
	δ	MaxDiff	δ	MaxDiff	δ	MaxDiff	δ	MaxDiff
N-be75np	0.8	2	0.85	1	0.95	1	0.925	2
N-t65w1lxx	0.915	2	0.989	1	0.957	1	0.989	1
N-t70d1lxx	0.915	2	0.957	1	0.968	1	0.957	1
N-t70f1lxx	0.733	4	0.8	3	0.867	2	0.9	1
N-t70w1lxx	1	0	1	0	1	0	1	0
N-t70x1lxx	1	0	0.75	1	1	0	1	0
N-t74d1lxx	0.956	2	0.978	1	0.989	1	0.989	1
N-t75d1lxx	1	0	1	0	1	0	1	0
N-t75e1lxx	0.875	2	0.875	2	0.938	1	0.938	1
N-t75i1lxx	0.778	2	0.889	1	0.917	1	0.972	1
N-tiw56r54	0.873	2	0.976	2	0.992	1	0.944	1
N-usa79	1	0	1	0	1	0	1	0
Avg.	0.903	1.5	0.922	1.083	0.964	0.75	0.967	0.75

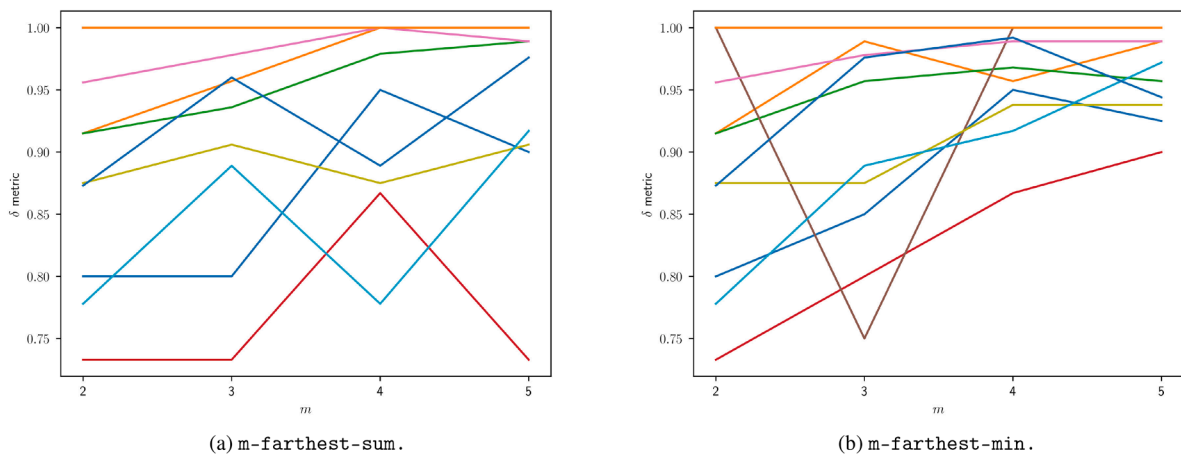


Fig. 6. Evolution of the δ metric for individual instances when m increases.

In Fig. 6, each line represents the δ -value for each individual instance. In the x -axis, each value of m permits us to compare the performance of δ in terms of this parameter. We can observe in the right image of the figure (representing the m -farthest-min model), a more consistent progression in this metric when increasing the value of m , than the m -farthest-sum model (represented in the left image), which exhibits an oscillation in some cases. In addition, we see that the δ met-

ric increases faster in the m -farthest-min model. This is in line with the results observed in the previous tables. In fact, with $m = 4$ these tables show that the accuracy of the m -farthest-min model is better than the m -farthest-sum model with $m = 5$. The best δ value overall is obtained with the m -farthest-min model with $m = 5$. In particular, it takes a remarkable $\delta = 0.967$. It must be noted that this model requires longer running times than the m -farthest-sum model. For example with $m = 3$ it

Table 8
Results of the VNS heuristic in the instances with optimal set $\mathcal{P}$ known.

Instance	$m = 2$		$m = 3$		$m = 4$		$m = 5$	
	δ	MaxDiff	δ	MaxDiff	δ	MaxDiff	δ	MaxDiff
N-be75np	0.8	2	0.975	1	1	0	0.975	1
N-t65w11xx	0.979	2	0.957	1	1	0	0.989	1
N-t70d11xx	0.915	2	0.957	1	1	0	0.979	1
N-t70f11xx	0.733	4	0.767	4	0.8	2	0.9	2
N-t70w11xx	1	0	1	0	1	0	1	0
N-t70x11xx	1	0	1	0	1	0	1	0
N-t74d11xx	0.956	2	0.967	2	1	0	0.978	2
N-t75d11xx	1	0	1	0	1	0	1	0
N-t75e11xx	0.688	2	0.75	2	0.906	2	0.969	1
N-t75i11xx	0.778	2	0.806	2	0.889	2	0.889	2
N-tiw56r54	0.873	2	0.976	2	0.913	4	0.984	1
N-usa79	1	1	1	0	1	0	1	1
Avg.	0.893	1.583	0.929	1.25	0.958	0.833	0.971	1

Table 9
Comparison of δ -measure averages.

	$m = 2$	$m = 3$	$m = 4$	$m = 5$
m-farthest-sum	0.903	0.928	0.944	0.950
m-farthest-min	0.903	0.922	0.964	0.967
VNS	0.893	0.929	0.958	0.971

requires between 10 and 25 minutes, while the m-farthest-sum model requires less than 5 minutes.

5.3. Heuristic algorithm

In the previous experiments, we have seen that it is a good strategy to compute three or four distant solutions to approximate the range of the positions of the elements in the entire set of optimal solutions, which is usually difficult to obtain. This can provide the user with useful information to interpret an optimal solution of the LOP in terms of its representativeness. However, as shown in previous experiments, even when considering instances for which the full set of optimal solutions can be computed, typically regarded as relatively easy, the running time of our mathematical models increases significantly as m increases. This is why we complement our proposal with the inclusion of a heuristic algorithm that provides high-quality solutions in short computational times. Table 8 reports on the results of our VNS heuristic in this data set.

After preliminary experimentation, we set the values of the VNS key-search parameters as $k_{max}^1 = 5$, $it_{max} = 1000$, $k_{max}^2 = \frac{n}{2}$. We run the method for $\tau_{max} = 15$ seconds. As in the previous experiment, we compare the VNS results with the entire range of the elements in $\mathcal{P}$ with the two statistics δ and MaxDiff. We run our heuristic with the parameter $m = 2, 3, 4$ and 5 to obtain the number of distant solutions respectively. This table shows that the heuristic is able to obtain very good results in all the cases, with average δ ranging from 0.893 to 0.971 respectively. Similarly, the average MaxDiff ranges from 1.583 to 0.833, indicating a good performance as well.

In Table 9, we summarize the results of the proposed methods, namely m-farthest-sum, m-farthest-min models, and VNS heuristic. It shows the δ statistic obtained with the three methods run with several values of m (from 2 to 5). The first observation is that all the results are very good, with values of δ close to 1. We can confirm the pattern observed in the previous experiment that the quality of the results (i.e., the approximation to the true range of elements) increases when m does. It is impressive the good results of the heuristic considering its low running times (15 seconds) compared with the models (larger than 20 minutes in some instances with $m = 5$). Interestingly, there are a few instances in which the heuristic provides a better approximation to the range of the elements in the set $\mathcal{P}$ than the mathematical models. This can be mis-

leading since generally speaking, mathematical models provide a lower bound of the heuristics in minimization problems. However, in our context, the heuristic may not find the optimal value of the problem in the first step, and thus it searches distant solutions in a suboptimal space, to approximate the true range in $\mathcal{P}$. Note that, in terms of the objective function values, the heuristic and the m-farthest-min model cannot be compared, and their accuracy cannot be directly compared. All the results are available in Benito et al. (2024).

We now perform an additional experiment to evaluate the VNS heuristic with larger values of m in the most difficult instances (i.e., those for which we cannot compute the set $\mathcal{P}$ explicitly). In particular, we consider the 21 I/O instances in our testbed for which the set $\mathcal{P}$ has more than 500,000 alternate optimal solutions. Note that we cannot compute the exact value of δ and MaxDiff since we do not know the range of positions of the elements in the entire set $\mathcal{P}$; so we approximate it with the largest range found across the different methods tested. The results shown in Table 10 confirm that in general terms, when our heuristic is run with larger values of m , it provides a better approximation of the range of each element in $\mathcal{P}$. Specifically, the VNS heuristic with $m = 5$ obtains an average value of $\delta = 0.889$ and with $m = 15$ results in an improved value of $\delta = 0.931$. Both values compare favorably with the previous model by Cameron et al. (2021) that obtains $\delta = 0.877$. A similar pattern is observed with the MaxDiff metric.

6. A case study. The premier soccer league

In this section, we apply the methodology proposed in this paper to analyze a practical problem. Specifically, we examine the results of the 2017–2018 Premier League soccer season. This is one of the most important applications of the LOP according to the monography of Martí and Reinelt (2022). The usual form for ranking soccer teams from the results of a league, is to reward each team with 3 points for each win, and 1 point for each tie; and then order them according to the total points. This system has three potential limitations. First, the lack of nuance, the current system does not differentiate between wins against strong or weak opponents. A victory against the top team worths the same than to the bottom team. Second, it does not directly account for the margin of victory, which can be an indicator of the team strength. Third, a team that consistently ties might accumulate more points than a team with a mix of wins and losses, even if the latter has shown the ability to decisively beat opponents. These reasons make reasonable to rank the teams using an LOP to provide a more representative ranking of the teams.

To approach the team's classification as an LOP, we need to define a dominance matrix. Since 20 teams participate in the Premier League, we create a 20x20 matrix where each row and column represents an individual team. In this matrix, the value of the entry (i, j) is set as the number of goals that team i scored against team j in the entire season.

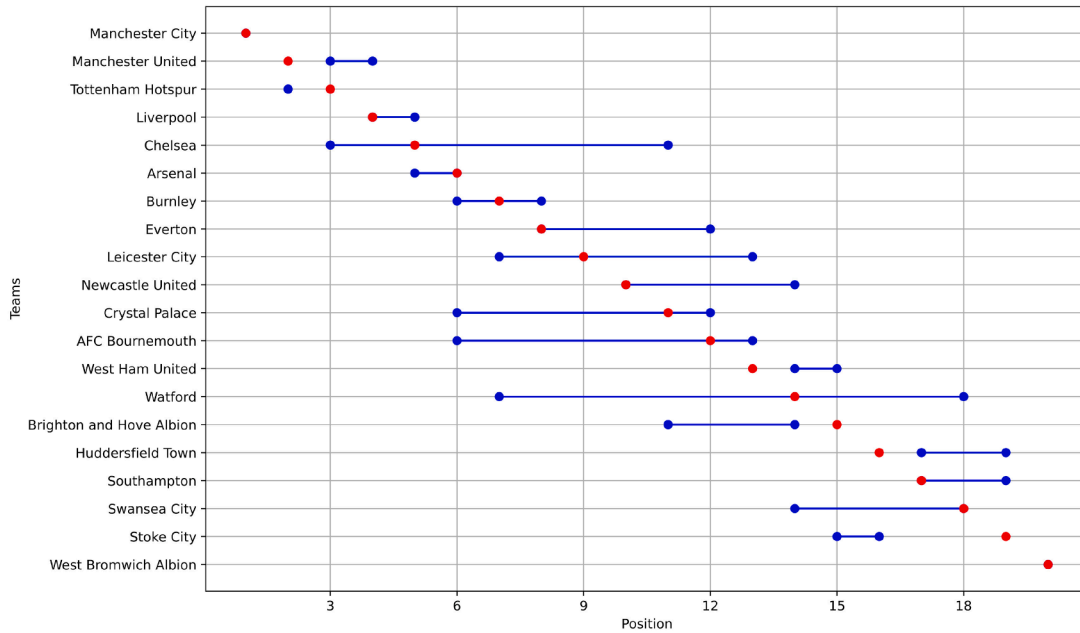


Fig. 7. Ranges of teams in LOP optimal solutions (Δ^P in blue) versus official classification (in red points). (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.).

Table 10
Comparison of the model and heuristic on the instances with $\mathcal{P}$ unknown.

Instance	Cameron $m = 2$		VNS $m = 5$		VNS $m = 10$		VNS $m = 15$	
	δ	MaxDiff	δ	MaxDiff	δ	MaxDiff	δ	MaxDiff
N-be75eec	0.852	4	0.935	5	0.974	3	1.000	0
N-be75oi	0.792	7	0.891	4	1.000	0	0.989	3
N-stabu70	1.000	0	0.725	17	0.721	16	0.736	16
N-t59b11xx	1.000	0	0.976	4	0.983	5	0.998	3
N-t59d11xx	1.000	0	0.814	7	0.866	7	0.927	5
N-t59f11xx	1.000	0	0.931	6	0.965	5	0.911	2
N-t59i11xx	0.719	12	0.781	8	0.908	5	1.000	0
N-t59n11xx	0.903	14	0.853	5	1.000	0	0.967	2
N-t65b11xx	0.552	4	0.936	3	1.000	0	0.985	2
N-t65d11xx	0.715	6	0.963	2	1.000	0	0.798	3
N-t65l11xx	0.695	14	1.000	0	0.978	8	0.984	5
N-t65n11xx	0.883	4	0.813	4	1.000	0	0.887	2
N-t69r11xx	1.000	0	0.761	6	0.976	3	0.926	4
N-t70b11xx	1.000	0	0.862	5	0.970	3	0.935	2
N-t70k11xx	0.744	5	1.000	0	0.806	3	0.787	3
N-t70l11xx	0.756	12	1.000	0	0.971	7	0.966	2
N-t70n11xx	0.996	6	0.780	3	0.901	3	1.000	0
N-t70u11xx	1.000	0	0.988	4	0.949	6	0.910	2
N-t75k11xx	0.861	6	1.000	0	0.904	2	0.939	3
N-t75n11xx	0.959	8	0.692	4	0.769	2	1.000	0
N-t75u11xx	1.000	0	0.969	2	0.910	4	0.906	4
Avg.	0.877	4.86	0.889	4.24	0.931	3.90	0.931	3.00

For example, if we associate Manchester City with the first row and column, and Manchester United with the second one, and they faced each other twice resulting in scores of 2–3 and 2–1 respectively, then the entry (1, 2) will be 4, and the entry (2, 1) will be 4 as well. We aim to maintain simplicity in this case study while demonstrating that computing three solutions is more representative than computing only two. To obtain these solutions, we employ the VNS algorithm with a runtime of 5 seconds. To facilitate comparison of results, we calculate the exact position ranges using the pool search method in GUROBI. Fig. 7 illustrates the official classification of each team with red dots and the exact range of positions that each team occupies across the optimal solution set with blue lines.

As anticipated, we observe that the team positions resulting from the LOP often differ significantly from the official classification. In some

cases, such as Huddersfield Town and Tottenham Hotspur, the official position falls outside the range of positions derived from the LOP, suggesting that the current ranking system may not accurately reflect these teams' performance. While there is a correlation between the official ranking and the LOP results, the analysis reveals that in the mid-table positions, the official ranking does not always reflect clear dominance between teams. It is noteworthy that some teams, such as Manchester City, demonstrated consistent dominance across all facets, ranking first in all the optimal solutions of the LOP. These findings highlight the potential of the LOP approach to provide a more nuanced evaluation of team performance, particularly in cases where traditional ranking methods may oversimplify complex competitive dynamics.

Table 11 presents the two most distant optimal LOP solutions, while Table 12 displays the three most distant optimal LOP solutions. An initial

Table 11

Two most distant LOP optimal solutions for the Premier League 2017–2018.

Position	Optimal Solution 1	Optimal Solution 2
1	Manchester City	Manchester City
2	Tottenham Hotspur	Tottenham Hotspur
3	Manchester United	Manchester United
4	Liverpool	Liverpool
5	Arsenal	Arsenal
6	Crystal Palace	AFC Bournemouth
7	Leicester City	Chelsea
8	Burnley	Burnley
9	Watford	Everton
10	AFC Bournemouth	Newcastle United
11	Chelsea	Crystal Palace
12	Everton	Leicester City
13	Brighton and Hove Albion	Brighton and Hove Albion
14	Newcastle United	Swansea City
15	West Ham United	West Ham United
16	Stoke City	Stoke City
17	Southampton	Huddersfield Town
18	Swansea City	Watford
19	Huddersfield Town	Southampton
20	West Bromwich Albion	West Bromwich Albion

Table 12

Three most distant LOP optimal solutions for the Premier League 2017–2018.

Position	Optimal Solution 1	Optimal Solution 2	Optimal Solution 3
1	Manchester City	Manchester City	Manchester City
2	Tottenham Hotspur	Tottenham Hotspur	Tottenham Hotspur
3	Manchester United	Manchester United	Chelsea
4	Liverpool	Liverpool	Manchester United
5	Arsenal	Arsenal	Liverpool
6	AFC Bournemouth	Crystal Palace	Arsenal
7	Chelsea	Leicester City	Burnley
8	Burnley	Burnley	Everton
9	Everton	Watford	Crystal Palace
10	Newcastle United	AFC Bournemouth	Leicester City
11	Crystal Palace	Chelsea	Brighton and Hove Albion
12	Leicester City	Everton	Watford
13	Brighton and Hove Albion	Brighton and Hove Albion	AFC Bournemouth
14	Swansea City	Newcastle United	Newcastle United
15	West Ham United	West Ham United	West Ham United
16	Stoke City	Stoke City	Stoke City
17	Huddersfield Town	Southampton	Southampton
18	Watford	Swansea City	Swansea City
19	Southampton	Huddersfield Town	Huddersfield Town
20	West Bromwich Albion	West Bromwich Albion	West Bromwich Albion

examination of the two most distant solutions might suggest that the top five teams are consistently ranked. However, when considering the three most distant solutions, we observe that Manchester United, Liverpool, and Arsenal interchange their positions, and Chelsea notably rises from 11th to 3rd position, thus disauthorizing the information provided with the two optimal solutions. This variability strongly supports the central thesis of our paper. Quantitatively, the δ metric for the two farthest optimal solutions is 0.65, while for the three farthest optimal solutions, it increases to 0.875, which represents a significant improvement. These results, derived from a real-life example, together with the statistical results described in the previous section, provide compelling evidence in support of our main argument regarding the importance of multiple optimal solutions in the LOP, and the convenience of providing the user with 3 or 4 distant optimal solutions, as opposed to one or two typically done in the existing literature.

7. Conclusions and future research

The linear ordering problem is a very well-known combinatorial optimization problem that has been used to solve a wide range of applications. Very little attention has been paid to the multiple optimal solutions that this problem usually has. We first identified the problem of

many several optimal solutions (rankings) with different orderings, and then proposed different solutions to overcome this situation in practical applications. In particular, we propose mathematical models and a heuristic method to compute a small number m of distant optimal solutions to approximate the range of the positions of the elements in the entire set of optimal solutions. In this way, we provide the user with a practical tool to disclose if the optimal solution considered provides or not a robust ranking.

It must be noted that the LOP has been applied in many domains of Science, ignoring the existence of multiple optimal solutions. We hope that this work motivates the researchers applying the LOP to study the similarity among optimal solutions and to moderate their conclusions based on a single ranking. Our study is limited to the I/O matrices since they permit to obtain all the optimal solutions to compute robust statistics, but we have to keep in mind that with larger instances, as considered in some domains, the effect of multiple optimal solutions may have more impact.

CRedit authorship contribution statement

Marc Benito-Marimon: Writing – original draft, Validation, Software, Investigation; **Rafael Martí:** Writing – review & editing, Writing – original draft, Supervision, Methodology, Investigation; **Anna Martínez-Gavara:** Writing – review & editing, Writing – original draft, Supervision, Methodology, Investigation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This research has been partially supported with grants PID2021-125709OB-C21 and PID2024-160226OB-C21 funded by MCIN/AEI/10.13039/501100011033 and by ERDF A way of making Europe. It has been also supported by the Generalitat Valenciana (CIAICO/2021/224).

References

- Anderson, P.E., Chartier, T.P., Langville, A.N., & Pedings-Behling, K.E. (2022). Fairness and the set of optimal rankings for the linear ordering problem. *Optimization and Engineering*, 23, 1289–1317. <https://doi.org/10.1007/s11081-021-09650-y>
- Benito, M., Martí, R., & Martínez-Gavara, A. (2024). Results for the multiple linear ordering problem. <https://data.mendeley.com/datasets/pjxp3d2vcj/2>.
- Cameron, T.R., Charmot, S., & Pulaj, J. (2021). On the linear ordering problem and the rankability of data. *Foundations of Data Science*, 3, 133–149. <https://doi.org/10.3934/fods.2021010>
- García, C.G., Pérez-Brito, D., Campos, V., & Martí, R. (2006). Variable neighborhood search for the linear ordering problem. *Computers & operations research*, 33(12), 3549–3565.
- Hansen, P., Mladenović, N., Todosijević, R., & Hanafi, S. (2017). Variable neighborhood search: Basics and variants. *EURO Journal on Computational Optimization*, 5(3), 423–454.
- Kemeny, J.G. (1959). Mathematics without numbers. *Daedalus*, 88(4), 577–591.
- Knuth, D.E. (1993). The Stanford GraphBase: A platform for combinatorial computing (vol. 1). ACM Press New York.
- Leontief, W. (1986). Input-output economics. Oxford University Press.
- Martí, R., Duarte, A., & Reinelt, G. (2010). LOLIB - Linear ordering problem library. <https://www.uv.es/rmarti/paper/lop.html>. [Online; accessed 29-April-2024].
- Martí, R., Martínez-Gavara, A., Sánchez-Oro, J., & Pérez-Peló, S. (2022). Discrete diversity and dispersion maximization. a review and an empirical analysis from an OR perspective. *European Journal of Operational Research*, 299 (3) *Invited review*, 795–813.
- Martí, R., & Reinelt, G. (2022). Exact and heuristic methods in combinatorial optimization. A study on the linear ordering and the maximum diversity problem (vol. 175). Springer.
- Mladenović, N., & Hansen, P. (1997). Variable neighborhood search. *Computers & Operations Research*, 24(11), 1097–1100.
- Parreño, F., Álvarez-Valdés, R., & Martí, R. (2021). Measuring diversity. A review and an empirical analysis. *European Journal of Operational Research*, 289(2), 515–532.